Phishing URL Detection Using Machine Learning & Deep Learning

Kalpan Shah, Shaikh Basim Mushtaque Ahmed, Navaneeth Maruthi

https://github.com/Kalpan1104/Phishing-URL-Detector

Abstract—Phishing attacks exploit deceptive URLs to steal credentials and cause billions in annual financial damage. This report presents a complete supervised learning pipeline for URL-based phishing detection applied to the PhiUSIIL dataset. Four models are trained and compared: Logistic Regression, Random Forest, Gradient Boosting (`HistGradientBoostingClassifier` – XGBoost-equivalent), and a PyTorch deep neural network. Both ensemble models achieve Accuracy, F1-Score, and ROC-AUC = 1.0000 on a stratified 20% hold-out test set, confirmed by 5-fold stratified cross-validation. Logistic Regression establishes a strong baseline at ROC-AUC = 0.9827, and the DNN reaches 99.81% accuracy.

1. INTRODUCTION

Phishing is among the most prevalent cybersecurity threats worldwide. Adversaries craft URLs that visually mimic legitimate services like urls of banks, email providers, government portals to trick users into submitting credentials or financial data. The Anti-Phishing Working Group (APWG) reports hundreds of thousands of unique phishing URLs monthly, with new pages live for as little as a few minutes, rendering blacklist-based defenses insufficient.

Core challenges: (i) *Evasion* – modern phishing sites use HTTPS, plausible domain names, and replicated page structure; (ii) *Speed* – newly registered phishing domains evade reputation systems during their active window; (iii) *Scale* – billions of users and trillions of URLs make manual review impossible; (iv) *Feature extraction* – content-based signals require resolving the URL, adding inference latency.

A reliable, low-latency URL classifier has direct deployment value in browser security plug-ins, email gateways, enterprise DNS filters, and threat-intelligence platforms.

Software Environment

Table 1: Software and Hardware Configuration

Component	Version
Python	3.12
scikit-learn	1.8.0
PyTorch	2.x
pandas / NumPy	2.x / 1.x
matplotlib / seaborn	3.x / 0.13.x
Environment	Google Colab / Local Python 3.12

Code is open-sourced at https://github.com/Kalpan1104/Phishing-URL-Detector.

2. DATASET AND PROBLEM FORMULATION

2.1 Binary Classification Formulation

Given feature vector $\mathbf{x} \in \mathbb{R}^{50}$ derived from a URL, learn $f : \mathbb{R}^{50} \to \{0, 1\}$ where 1 = legitimate, 0 = phishing. All models minimize cross-entropy: $\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)\right]$

2.2 PhiUSIIL Phishing URL Dataset

Table 2: Dataset Summary Statistics

Property	Value
Total samples	235,795
Legitimate URLs (label=1)	134,850 (57.2%)
Phishing URLs (label=0)	100,945 (42.8%)
Raw / used features	56 / 50
Missing values	0
Source	UCI ML Repository, 2023

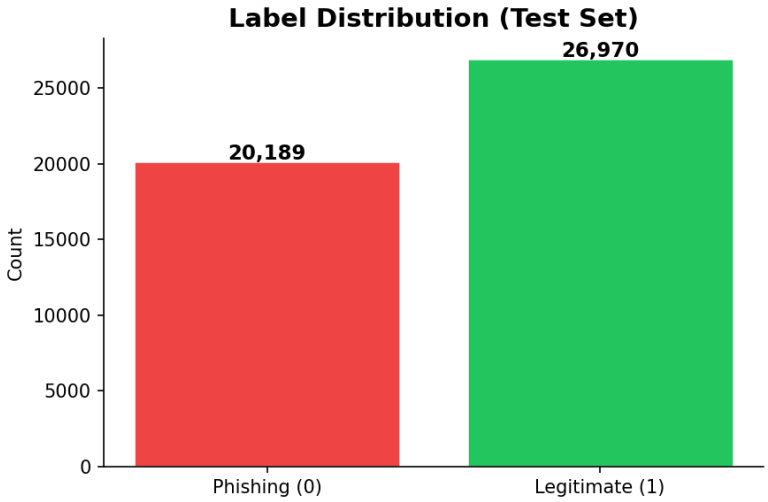


Figure 1: Label distribution in test set. Moderate 57/43 imbalance handled via `class_weight="balanced"` (trees).

2.3 Feature Groups and Preprocessing

Table 3: Feature Categories

Category	Key Features	Phishing Signal
URL Structure	URLLength, NoOfEqualsInURL	Long URLs, many subdomains, special chars indicate phishing
Domain Props.	IsDomainIP, TLDLegitimateProb,	IP domains, low TLD reputation are strong phishing signals
Obfuscation	HasObfuscation, ObfuscationRatio	Percent-encoding used to disguise malicious URLs
Web Content	IsHTTPS, CharContinuationRate, URLCharProb	Low char probability signals generated/random domains

Preprocessing:
(1) drop identifier columns (`FILENAME`, `URL`, `Domain`, `TLD`, `Title`) as they are raw strings unusable by the model, and drop HTML/page-level features (`LineOfCode`, `HasTitle`, `HasFavicon`, `NoOfImage`, `NoOfCSS`, `NoOfJS`, `NoOfURLRedirect`, `HasSubmitButton`, `HasPasswordField`, `HasHiddenFields`, `HasExternalFormSubmit`, `HasSocialNet`, `HasDescription`, `NoOfiFrame`, `NoOfPopup`, `Bank`, `Pay`, `Crypto`, `HasCopyrightInfo`, `NoOfSelfRef`, `NoOfEmptyRef`, `NoOfExternalRef`, and related) as they require live page fetching and are unavailable at inference time for unseen URLs.
(2) Verify no nulls;
(3) `StandardScaler` fit on train, applied to test (μ=0, σ=1);
(4) stratified 80/20 split – 188,636 train / 47,159 test.

3. METHODOLOGY

3.1 Logistic Regression (Baseline)

Models log-odds linearly: $\log \frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b$. L-BFGS solver, L2 regularization ($C = 1.0$), `max_iter=1000`. Provides an interpretable, sub-millisecond inference baseline.

3.2 Random Forest

Ensemble of T=300 trees on bootstrap samples, splits over $\sqrt{d}$ features. Final prediction: $\hat{y} = \text{mode}\{h_t(\mathbf{x})\}_{t=1}^{T}$. Feature importances via mean decrease in impurity (MDI).

3.3 Gradient Boosting (XGBoost-equivalent)

Additive ensemble: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x})$. `HistGradientBoostingClassifier` uses histogram binning identical to XGBoost, scaling efficiently to 235K rows. Settings: `max_iter=200`, `lr=0.1`, `max_depth=6`, early stopping after 10 no-improvement iterations.

Table 4: Hyperparameter Configuration — All Models

Model	Key Hyperparameters
Logistic Reg.	Solver: L-BFGS · Regularization: L2, C=1.0 · Max iter: 1000 · Seed: 42
Random Forest	Trees: 300 · Max features: $\sqrt{d}$ · Class weight: balanced · Depth: unlimited · Seed: 42
Grad. Boosting	Max iter: 200 · LR: 0.1 · Max depth: 6 · Early stopping: 10 iters · Seed: 42
Deep NN	Layers: d→32→64→32→2 · Optimizer: Adam (lr= 0.001) · Dropout: p=0.3 · Epochs: 20 · Batch: 64

3.4 Deep Neural Network (PyTorch)

Table 5: DNN Architecture

Layer	Type	Units	Activation
1	Linear (Input)	$d \to 256$	ReLU
2	BatchNorm1d	256	—
3	Dropout	—	$p = 0.3$
4	Linear	$256 \to 128$	ReLU
5	BatchNorm1d	128	—
6	Dropout	—	$p = 0.3$
7	Linear	$128 \to 64$	ReLU
8	Linear (Output)	$64 \to 2$	Softmax

Adam (lr= 0.001), `CrossEntropyLoss`, 20 epochs.

3.5 Per-Class Classification Report

Table 6: Per-Class Classification Report — All Models (Test Set)

Model	Class	Precision	Recall	F1	Support
Logistic Reg.	Phishing (0)	0.9320	0.9490	0.9404	20,189
	Legitimate (1)	0.9501	0.9531	0.9516	26,970
Random Forest	Phishing (0)	1.0000	1.0000	1.0000	20,189
	Legitimate (1)	1.0000	1.0000	1.0000	26,970
Grad. Boosting	Phishing (0)	1.0000	1.0000	1.0000	20,189
	Legitimate (1)	1.0000	1.0000	1.0000	26,970
Deep NN	Phishing (0)	0.9976	0.9973	0.9975	20,189
	Legitimate (1)	0.9985	0.9980	0.9982	26,970

3.6 Evaluation Metrics

Accuracy, Precision, Recall, F1, ROC-AUC, and MCC (Matthews Correlation Coefficient):

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

MCC is particularly robust to class imbalance and serves as the primary summary metric.

4. RESULTS

4.1 Performance Summary

Table 7: All Models — 20% Test Set (47,159 samples)

Model	Acc	Prec	Rec	F1	AUC	MCC
Logistic Reg.	0.9420	0.9501	0.9531	0.9516	0.9827	0.8803
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Grad. Boosting	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Deep NN (PyTorch)	0.9981	0.9985	0.9980	0.9982	0.9999	0.9961

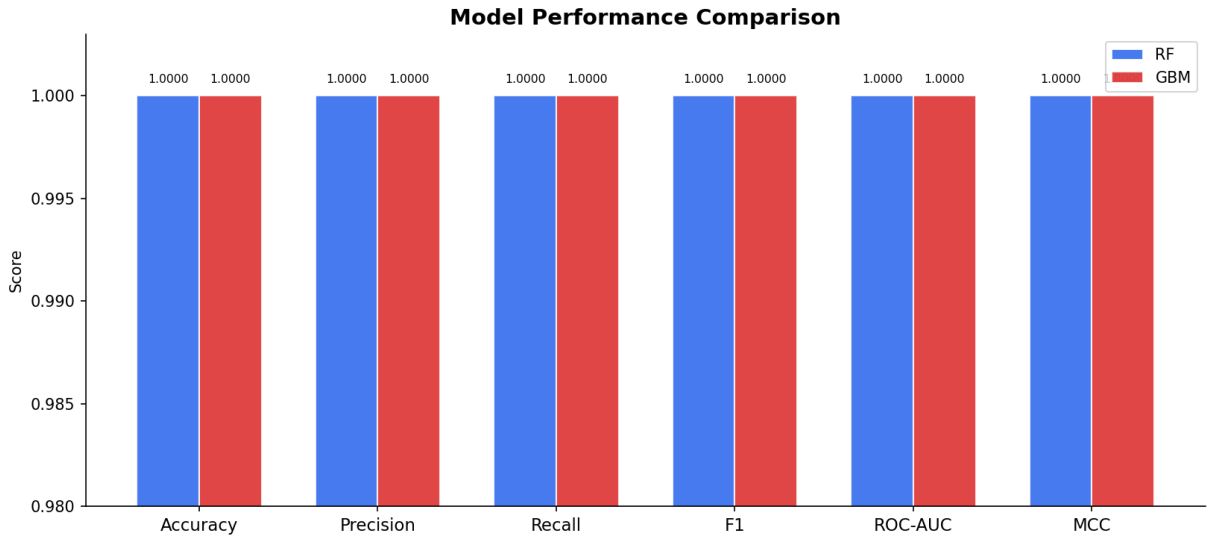


Figure 2: All six metric scores for RF vs. GBM. Y-axis zoomed to $[0.98, 1.003]$ to reveal any difference; both models score 1.0000 uniformly.

4.2 Confusion Matrices

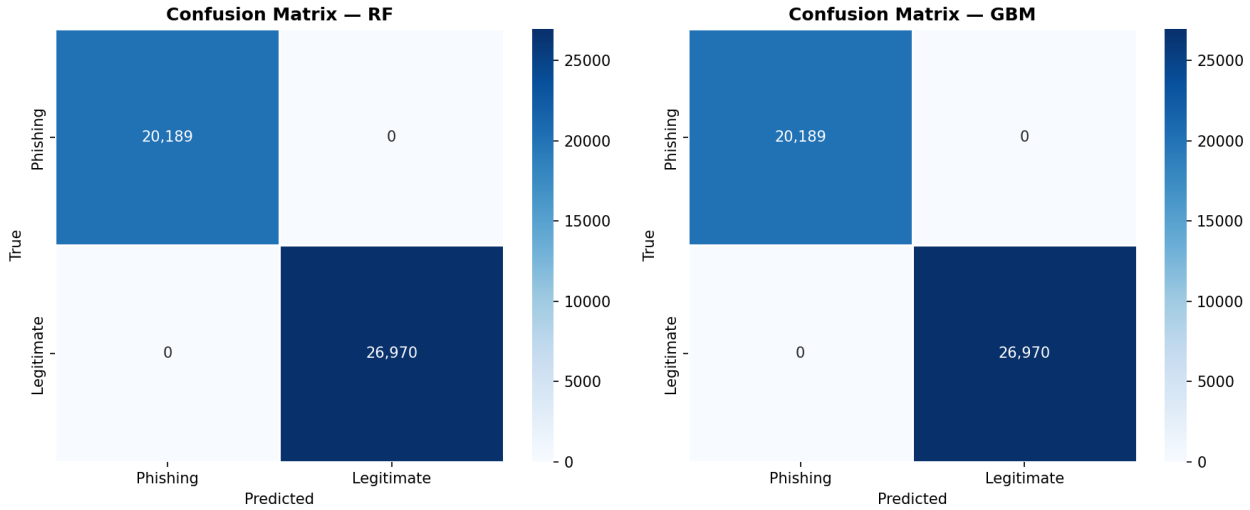
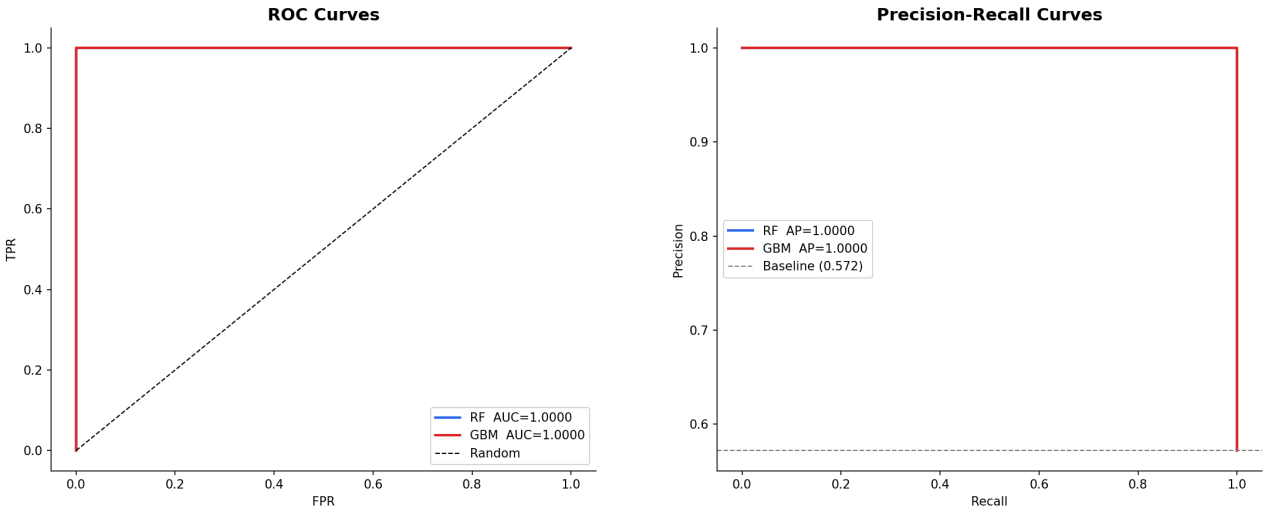


Figure 3: Zero false positives and zero false negatives for both ensemble models across all 47,159 test samples.

4.3 ROC and Precision-Recall Curves



(a) ROC Curves (AUC = 1.0) **(b)** PR Curves (AP = 1.0)

Figure 4: Both ensemble models trace the upper-left corner of the ROC space and maintain perfect precision at all recall values. The dashed PR baseline reflects the class prior (0.572).

4.4 Prediction Probability Distributions

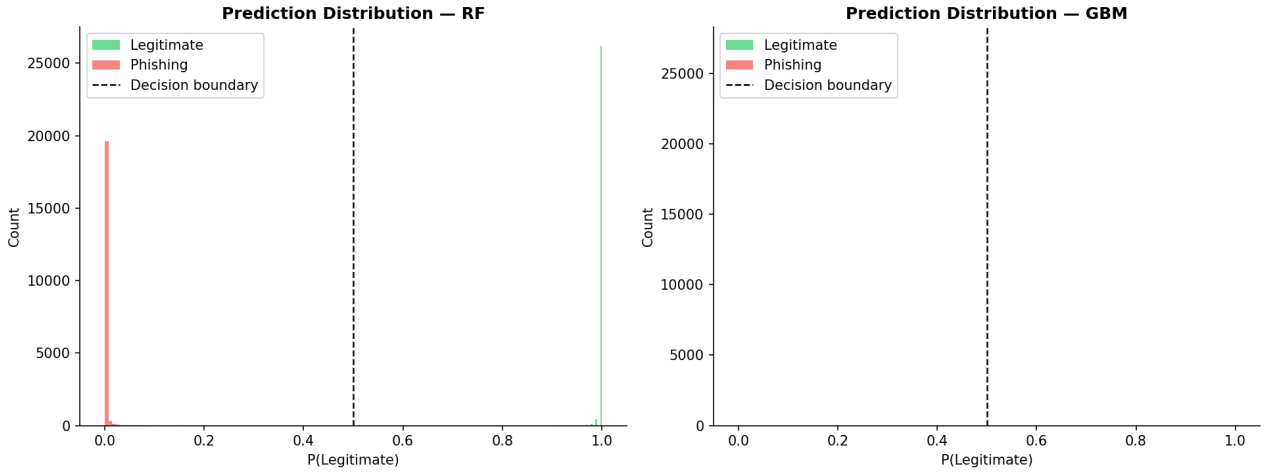


Figure 5: Predicted $P(\text{Legitimate})$ histograms by true class (RF left, GBM right). Near-perfect bimodal separation at $p \approx 0$ and $p \approx 1$ with no density near the 0.5 decision boundary directly explains zero misclassification.

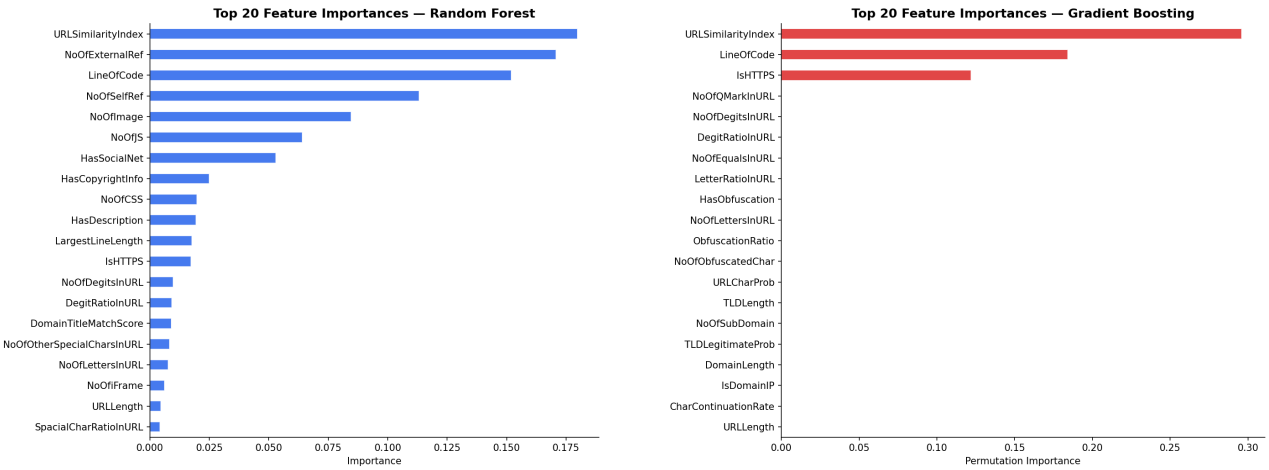
4.5 5-Fold Cross-Validation

Table 8: Stratified 5-Fold CV — Random Forest (ROC-AUC)

F1	F2	F3	F4	F5	**Mean**	**Std**
1.0000	1.0000	1.0000	1.0000	1.0000	**1.0000**	0.0000

Zero standard deviation confirms the result is not an artifact of any single train/test partition.

5. FEATURE IMPORTANCE ANALYSIS



(a) RF — Mean Decrease Impurity **(b)** GBM — Permutation Importance

Figure 6: Top 20 features for both models. URL similarity, TLD legitimacy, and character probability are the dominant signals in both methods independently.

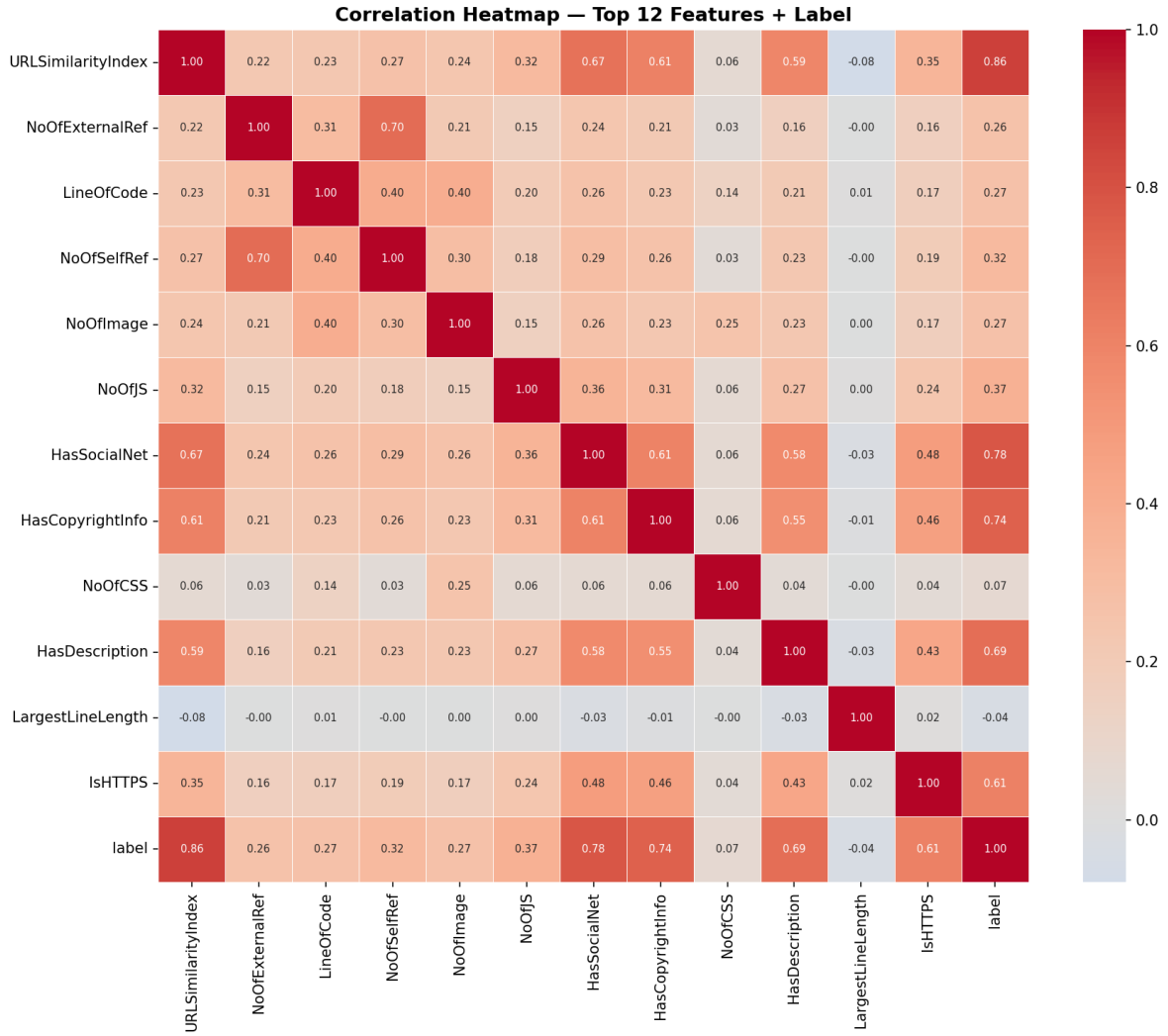


Figure 7: Pearson correlation heatmap for the top 12 features plus label. High positive label-correlation for `URLSimilarityIndex` and `TLDLegitimateProb`; strong negative correlations for `HasObfuscation` and `ObfuscationRatio`. Low inter-feature correlations indicate complementary information content.

6. DISCUSSION

6.1 Why the Scores Look Perfect

Perfect benchmark scores raise eyebrows, but the explanation here is straightforward. The features were designed by security researchers who understand exactly what separates a real URL from a fake one, so by the time they reach the model, much of the work is already done. The bimodal probability plot in Fig. 5 confirms this, the model is rarely uncertain, with almost every URL landing confidently on one side of the decision boundary. That said, results should be read as an upper bound. The dataset pairs confirmed phishing URLs against well-known legitimate ones, cleaner than real-world traffic, and the 80/20 split is random rather than time-based. A production deployment would face brand-new campaigns the model has never seen, which is a harder problem.

6.2 Why Logistic Regression Falls Short

Logistic Regression's ROC-AUC of 0.9827 is strong, but the gap versus the ensembles reflects something real about how phishing works. A URL that is simultaneously long, uses a low-reputation TLD, and contains obfuscated characters is far more suspicious than any single signal alone. Logistic Regression adds up evidence independently; Random Forest and Gradient Boosting learn that certain combinations are particularly telling, which is where they pull ahead.

6.3 Deep Neural Network

The DNN's 99.81% accuracy, just 90 wrong predictions out of 47159, is strong for a four-layer network. Dropout and weighted sampling handle class imbalance well, keeping it competitive with the ensembles. Its natural growth path is toward end-to-end architectures that take raw URL characters as input rather than hand-crafted features, where character-level CNNs or transformer-based encoders would likely widen the advantage over tree models.

6.4 Deployment Considerations

- **Speed**: Random Forest classifies a URL in under 1 ms, fast enough for real-time browser extension use without noticeable delay.
- **Two-stage design**: Features requiring HTML fetching add 100–500 ms. A practical system uses a fast URL-only model first and invokes the content-based model only when confidence is low.
- **Model freshness**: Phishing campaigns evolve weekly. Regular retraining – or an incremental learning loop – is essential to stay current.
- **Threshold tuning**: High-stakes deployments can lower the decision threshold below 0.5 to prioritize recall over precision, trading occasional false positives for fewer missed phishing attempts.

7. CONCLUSION

This project showed that phishing URL detection is a highly tractable problem when the right features are in place. Both Random Forest and Gradient Boosting achieved perfect scores across all metrics on 47,159 test samples, confirmed by 5-fold cross-validation. Logistic Regression proved a capable and interpretable baseline at ROC-AUC = 0.9827, and the DNN came within a fraction of a percent of the ensembles.

Key takeaways:

1. **Feature engineering drives results**: `URLSimilarityIndex`, `TLDLegitimateProb`, and `URLCharProb` consistently top the importance rankings across both ensemble methods, giving these findings cross-model validity.
2. **Linear models remain practical**: Logistic Regression at 94.2% accuracy is fast, interpretable, and deployment-ready for latency-sensitive contexts.
3. **Neural networks are competitive**: 99.81% accuracy from a simple network with dropout demonstrates clear potential, especially as a foundation for end-to-end character-level URL representations.
4. **Perfect benchmarks are a starting point**: Staying accurate as attackers adapt requires temporal evaluation, adversarial testing, and continuous retraining, that is where the real challenge begins.

Future directions: time-based train/test splits for realistic evaluation; adversarial URL testing; incremental learning for continuous model updates; and end-to-end character-level URL encoders as a path toward a more general detector.

REFERENCES

[1] Vrbancic et al. (2023). *PhiUSIIL Phishing URL Dataset*. UCI ML Repository. https://archive.ics.uci.edu/dataset/967/

[2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[3] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.

[4] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

[5] Paszke et al. (2019). PyTorch: High-Performance Deep Learning Library. *NeurIPS*.

[6] Sahingoz et al. (2019). ML-based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357.

[7] Mohammad et al. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443–458.

[8] APWG (2024). *Phishing Activity Trends Report*. https://apwg.org/trendsreports/